

EPTQ: Factored E8로 2-bit LLM 양자화 읽기

EPTQ: Fast and Accurate 2-bit Post-Training Quantization via Factored E8 Lattice

Anonymous Author(s) (2026)

작은 코드북, 분포 정렬, 적응형 FP16 보존을 하나의 PTQ 흐름으로 연결한 공개 원고

먼저 확인할 공개 상태

익명 CIKM '26 템플릿의 공개 원고이며 최종 ACM 출판물로 단정할 수 없다

- 저자 표기는 Anonymous Author(s)다
- 첫 쪽 DOI와 ISBN은 placeholder로 표시된다
- 본문은 code와 model을 acceptance 뒤 공개한다고 쓴다
- 따라서 이 텍스트는 공개 manuscript의 주장과 조건을 설명한다

실험 결과는 공개 초안의 근거이며, 수락·배포·재현을 이미 완료한 증거는 아니다

2-bit PTQ의 출발점: 작은 코드와 빠른 조회를 함께 얻기

가중치 하나씩 반올림하는 대신 8개를 묶어 벡터 코드로 표현하지만, 큰 코드북은 다시 병목이 된다

01 학습 완료 LLM W를 재학습 없이 압축



02 8개 묶음 한 열의 연속 8개 행



03 벡터 코드 16-bit code 하나로 묶음 표현



04 직접 E8 표 65,536개 8-D 점은 1 MiB

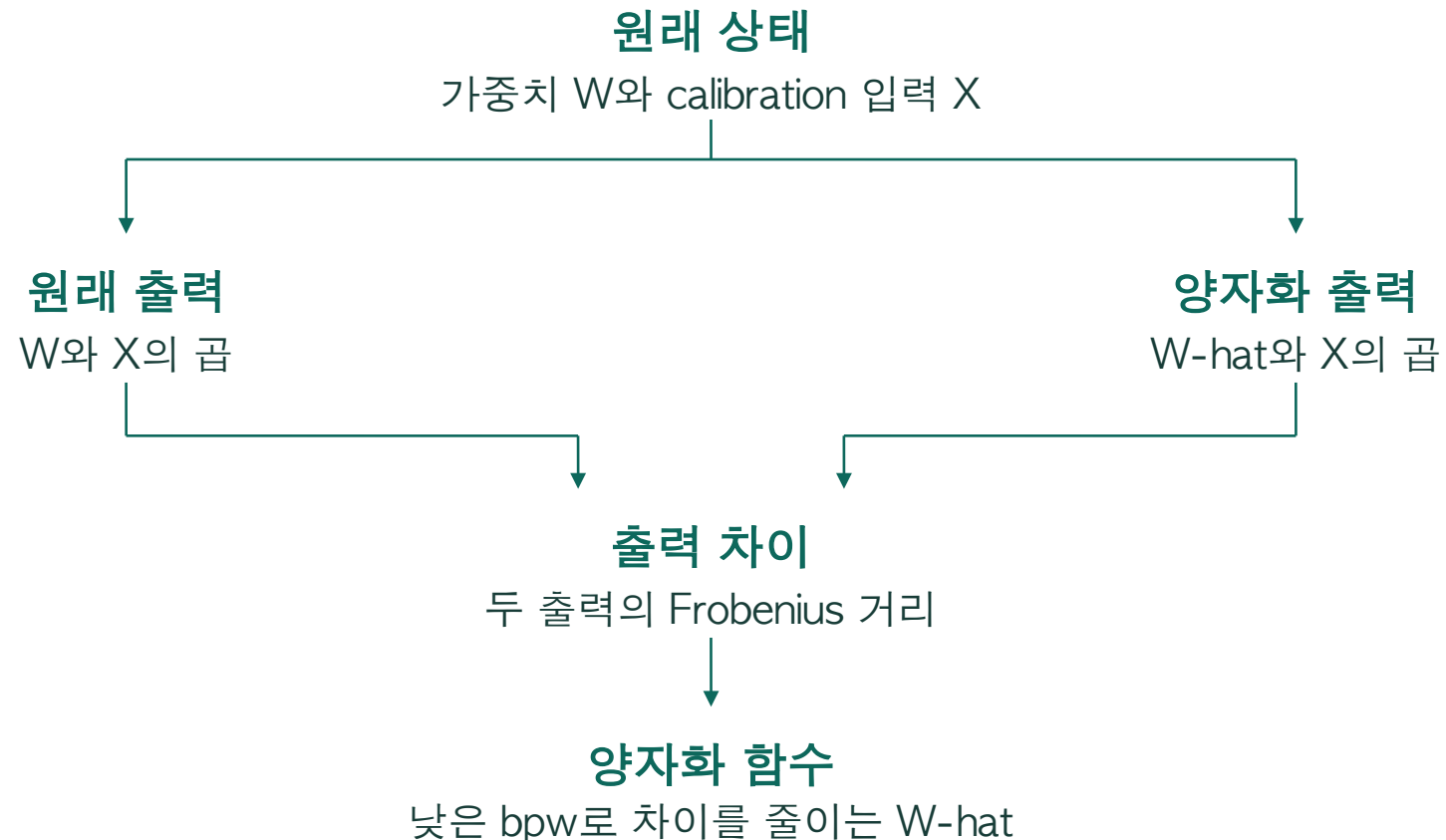


05 EPTQ 질문 점 수는 유지하며 표를 줄일 수 있나

EPTQ의 설계 문제는 단지 bit 수가 아니라 코드북이 cache에 머무를 수 있는가다

양자화 목표는 출력 변화가 작은 $\hat{W}$ 찾기

입력 X 에 대한 원래 출력과 양자화 출력의 차이를 작게 만들려 한다



양자화 단위는 한 column 안의 8-row vector

행 방향으로 8개를 잘라 같은 column j 의 벡터 v 를 만든다



이 방향을 알아야 column-wise compensation과 scale alpha의 의미를 올바르게 읽을 수 있다

4-D half의 type은 좌표 형식과 합의 parity로 정한다

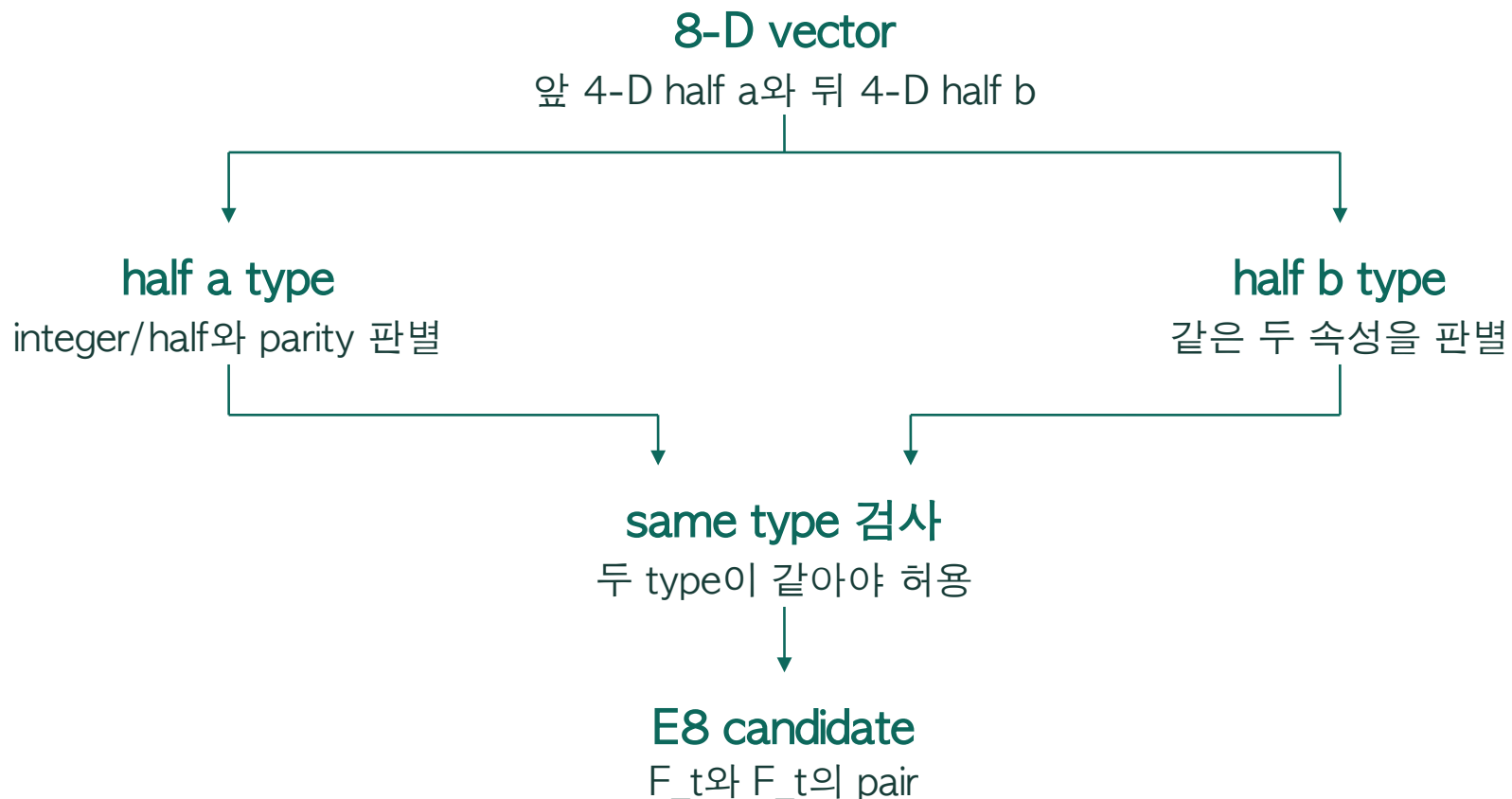
8-D E8 vector를 두 4-D half로 나눈 뒤, 각 half에 두 이진 속성을 붙인다

type	좌표	좌표합 parity	강의용 예
0	integer	even	(1,1,0,0)
1	half-integer	even	(.5,.5,.5,.5)
2	integer	odd	(1,0,0,0)
3	half-integer	odd	(-.5,.5,.5,.5)

type은 값의 크기가 아니라 integer 여부와 parity를 함께 기록한 분류다

E8 후보는 같은 type의 두 half만 결합한다

E8은 네 개의 same-type 4-D product의 서로 겹치지 않는 합으로 분해된다



강의용 type 조합: parity가 다르면 결합할 수 없다

아래 숫자는 원문 실험이 아닌 같은-type 규칙을 확인하는 작은 예다

앞 half	뒤 half	판정	이유
(1,1,0,0)	(2,0,0,0)	가능	둘 다 integer-even
(1,0,0,0)	(0,0,1,0)	가능	둘 다 integer-odd
(1,1,0,0)	(1,0,0,0)	불가	even과 odd가 다름
(.5,.5,.5,.5)	(-.5,.5,.5,.5)	불가	half-even과 half-odd

두 half가 모두 integer여도 parity가 다르면 같은 E8 product에 속하지 않는다

FE8: 512개 4-D vector로 65,536개 후보 만들기

각 type에서 원점에 가까운 128개 4-D point만 저장하고, 같은 type pair를 조합한다

01	4개 type	각 type을 따로 보관
↓		
02	128개씩	4-D representative 512개
↓		
03	same-type pair	type t 안에서 i와 j 결합
↓		
04	4×128^2	65,536개 8-D candidate
↓		
05	한 code	t, i, j를 16 bit에 pack

FE8은 후보 수를 줄인 것이 아니라 후보의 저장 표현을 factorization으로 바꾼다

코드북 table footprint 계산

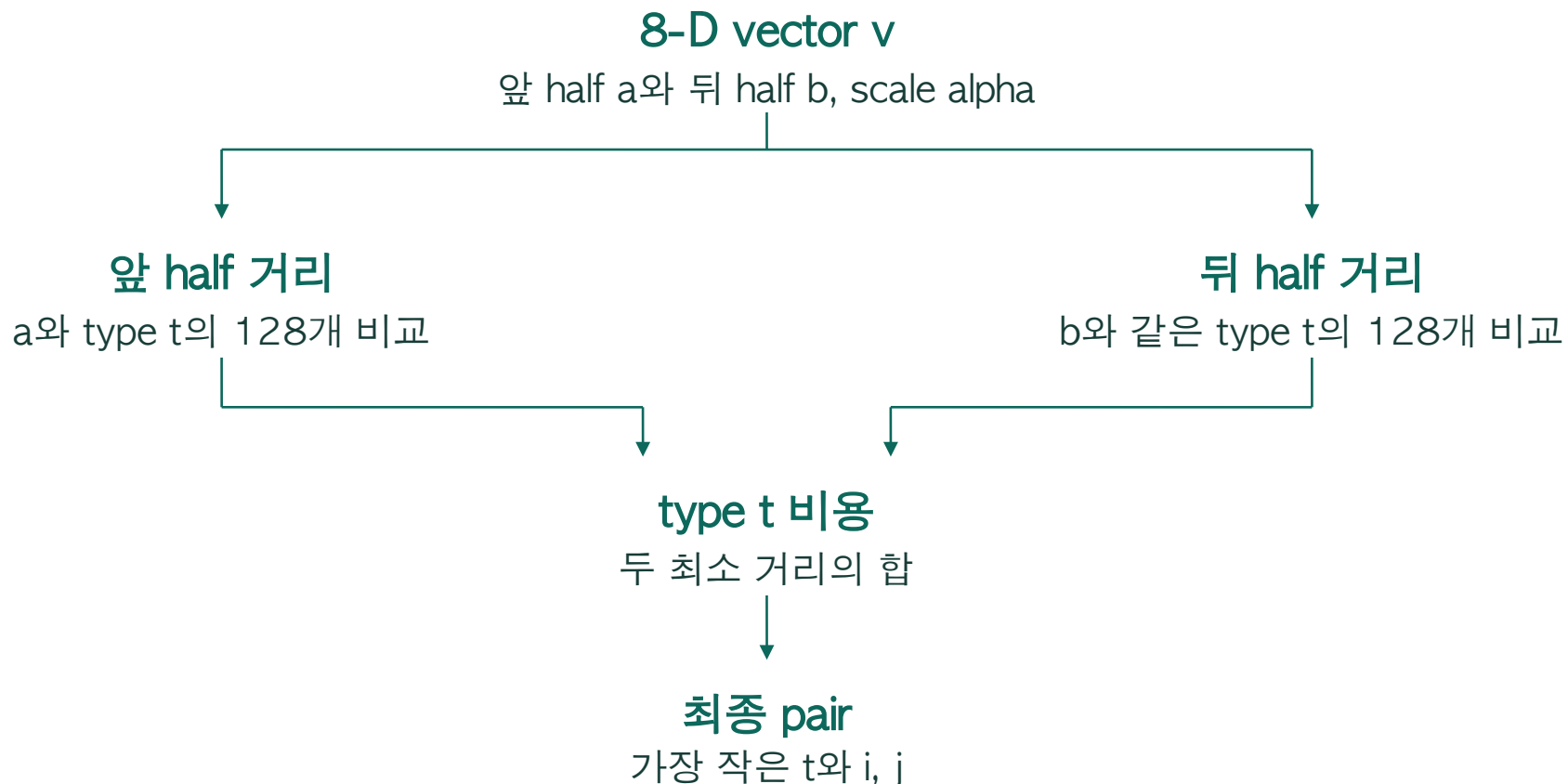
동일한 65,536개 8-D candidate 수에 대해 직접 저장과 FE8 저장은 다르다

표현	저장 대상	계산	크기
직접 E8	65,536개의 8-D FP16	$65,536 \times 8 \times 2 \text{ B}$	1 MiB
FE8	512개의 4-D FP16	$4 \times 128 \times 4 \times 2 \text{ B}$	4 KiB
비율	코드북 table만	1 MiB / 4 KiB	256×

256×는 코드북 table만의 footprint 비율이지 전체 model 저장·속도 비율이 아니다

FE8 search는 type별 두 half 거리의 합을 비교한다

4개 type마다 128개 후보와의 거리를 계산한 뒤, 한 type 안의 최적 pair를 고른다



1 6-bit code는 type과 두 subindex를 pack한다

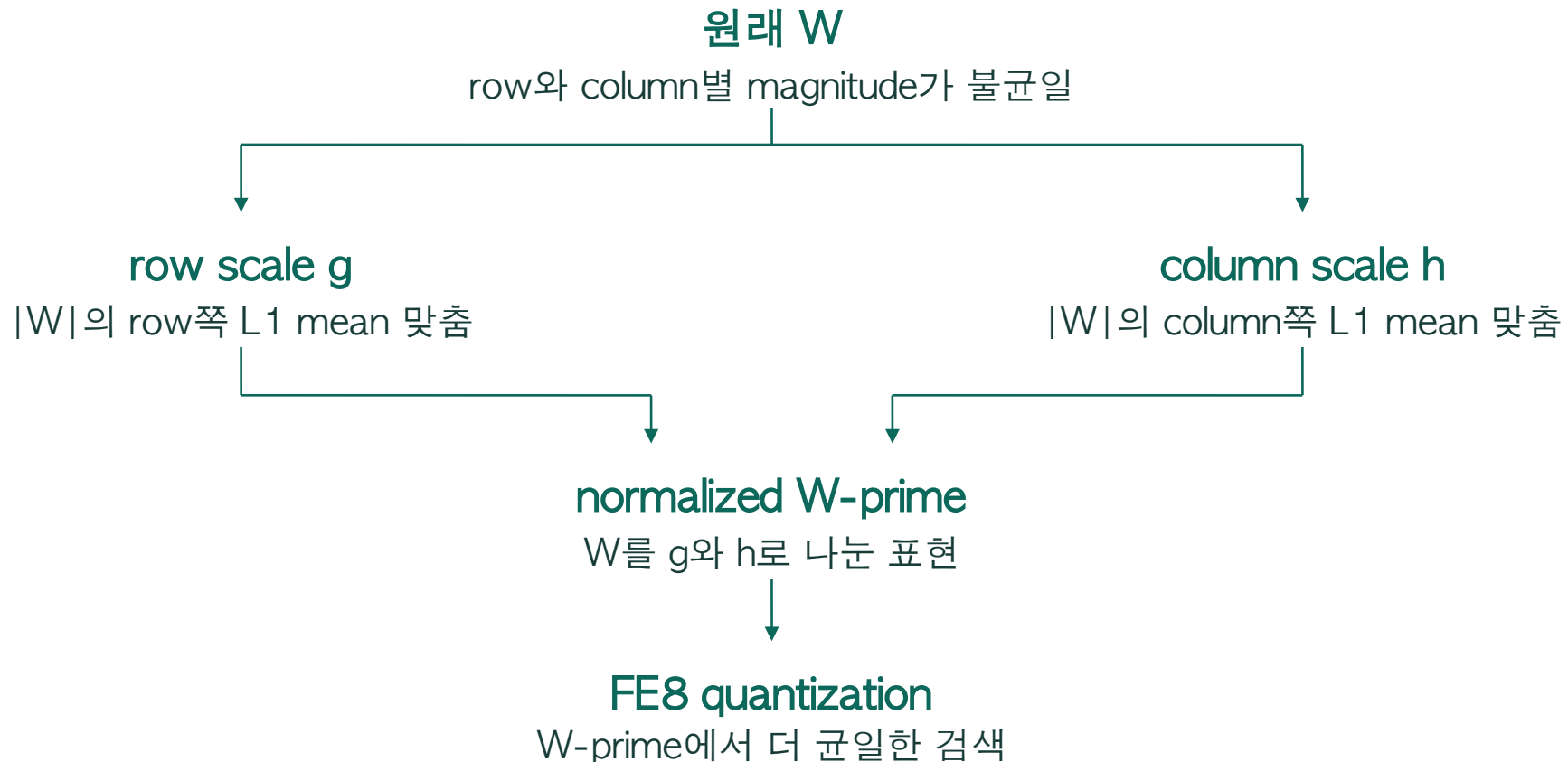
정상 code 하나가 type t 와 두 7-bit 위치 i, j 를 나타내며 마지막 code는 sentinel이다

01	type t	0부터 3까지
↓		
02	앞 index i	0부터 127까지
↓		
03	뒤 index j	0부터 127까지
↓		
04	강의용 계산	$t=2, i=3, j=5$ 이면 33,157
↓		
05	reserved code	65,535는 FP16 보존 포식

sentinel은 code 공간의 한 값을 쓰므로 normal quantization code 하나를 포기하는 설계다

WSN: quantization 전 scale 불균형 줄이기

가중치의 평균은 대체로 0에 가깝지만 row와 column의 크기는 크게 다를 수 있다



WSN의 alternating scale update

scale g 와 h 가 안정될 때까지 row와 column을 교대로 조정한다

01	초기화	g 와 h 를 all-ones로 둔다
↓		
02	row update	현재 h 로 row L1 mean을 맞춤
↓		
03	column update	현재 g 로 column L1 mean을 맞춤
↓		
04	수렴	scale factor가 안정될 때 종료
↓		
05	재표현	$W = W\text{-prime elementwise } (g \ h\text{-transpose})$

WSN은 reversible한 재매개화이며, 양자화 전 $W\text{-prime}$ 를 정돈하는 단계다

강의용 WSN 계산: scale을 곱해도 양자화 전 출력은 같다

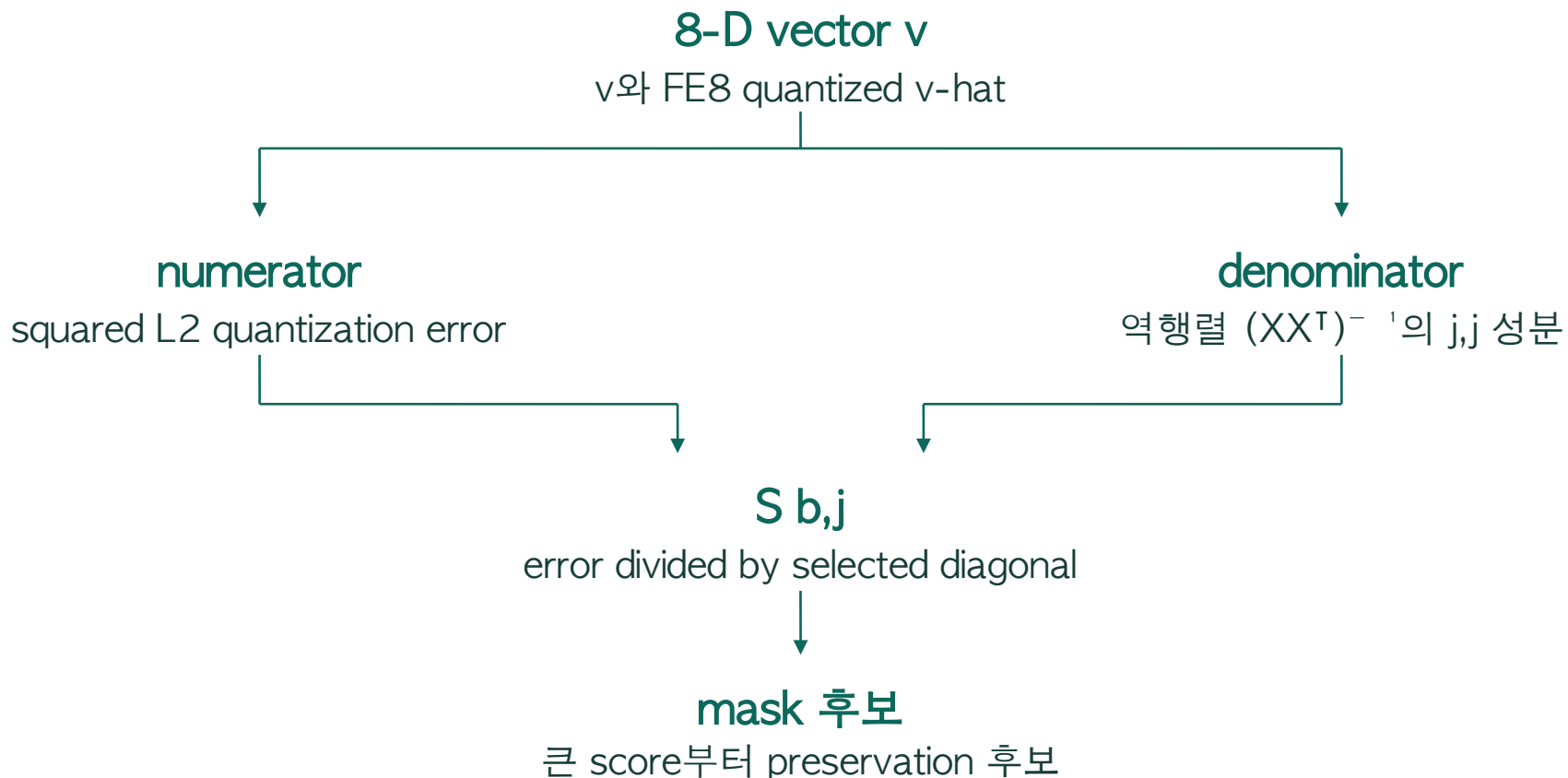
작은 2×2 예로 $W = \text{diag}(g)$ W-prime $\text{diag}(h)$ 를 직접 확인한다

항	값	계산 결과
W	$\{[2,4],[1,2]\}$	$x = [1,2]$ 이면 $Wx = [10,5]$
g	$[3,1.5]$	row scale
h	$[2/3, 4/3]$	column scale
W-prime	all ones	$\text{diag}(g)W\text{-prime} \text{diag}(h)x = [10,5]$

이 등식은 양자화 전 재표현의 동일성만 보이며, 양자화 뒤 품질 보장은 아니다

CWP score: 한 vector의 local loss proxy

제공 오차를 calibration Gram 역행렬의 j번째 대각 성분으로 나눈 local score다



rho와 Kneedle은 서로 다른 역할을 한다

rho는 score 곡선의 앞부분을 탐색할 크기이고, Kneedle이 matrix별 threshold tau를 선택한다

01	N scores	모든 8-D vector score
↓		
02	top K	$K = \max(2, \text{floor}(N \rho))$
↓		
03	정규화	rank와 log score를 0부터 1로
↓		
04	knee	곡선과 대각선 거리 최대
↓		
05	tau와 mask	S가 tau 이상이면 FP16 보존

rho가 10%여도 최종 보존 fraction f가 10%라는 뜻은 아니다

강의용 Kneedle 계산: $\rho=.1$ 이 $f=.1$ 은 아니다

$N=40$, top $K=4$ 인 constructed score 예로 threshold와 realized fraction을 분리한다

순위 k	score	정규 rank	정규 log score	거리
1	1000	0	1	0
2	10	1/3	1/3	1/3
3	2	2/3	.10034	.23299
4	1	1	0	0

threshold 10이면 40개 중 2개만 남아 $f=5\%$ 가 된다

sentinel은 mask만 없애고 FP16은 남긴다

normal FE8 block과 preserved block의 저장 단위를 구분해야 bpw를 해석할 수 있다

block	저장	8 weights 기준	bpw
normal	16-bit FE8 code	16 bits	2
preserved	sentinel + 8 FP16	16+128=144 bits	18
mask	별도 1-bit mask	sentinel로 대체	별도 mask 없음

zero bit-overhead는 별도 mask에 대한 말이며 preserved vector 자체가 무료라는 뜻은 아니다

Appendix B의 bpw는 f와 scale vector를 함께 센다

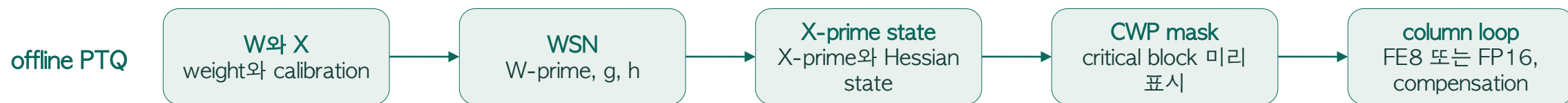
표시된 식은 normal 2 bpw, preservation cost, g와 h storage로 구성된다

- $\text{bpw} = 2 + 16f + 16(1/d\text{-row} + 1/d\text{-col})$
- f는 실제 preserved vector fraction이다
- 마지막 항은 FP16 g와 h vector의 matrix당 overhead다
- Appendix B는 per-column alpha의 storage를 명시적으로 세지 않는다

reported bpw는 f에 따라 변하고, alpha storage는 원고의 accounting 빈칸이다

EPTQ Algorithm 3: mask와 column compensation

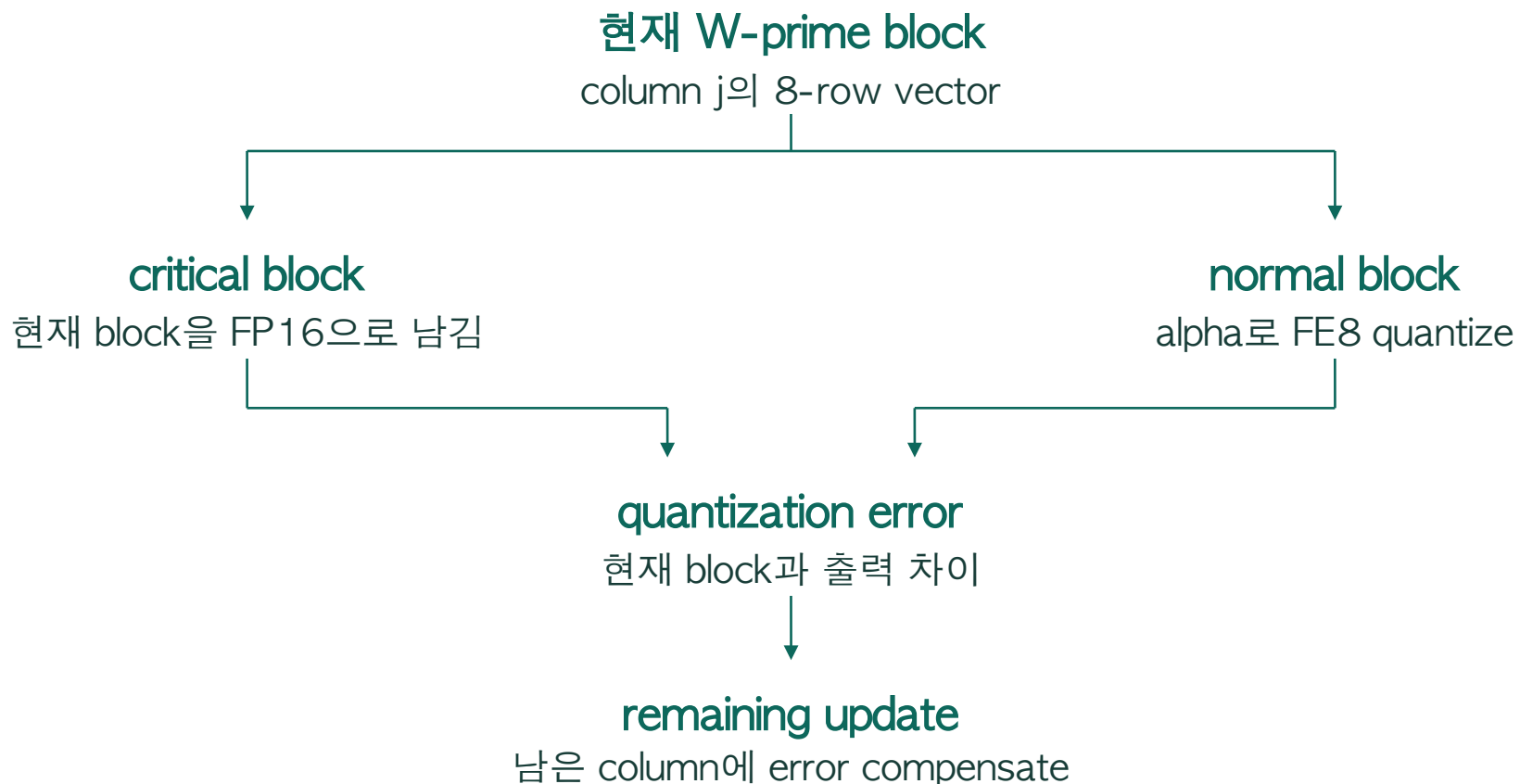
WSN 뒤에는 effective input으로 state를 만들고, column과 8-row block을 순서대로 처리한다



CWP mask는 quantization loop 전에 만들고, W-prime의 현재 상태를 순차적으로 갱신한다

block 처리: FP16 보존 또는 FE8 quantization

critical 여부에 따라 raw original W가 아니라 그 시점의 W-prime block을 사용한다



실험을 읽는 최소 조건

Table 1의 수치는 LLM, calibration, metric, hardware가 함께 정한 조건부 수치다

항목	원고 조건	해석 경계
모델	Llama-2 7B·13B·70B, Llama-3 8B	다른 architecture로 일반화 불가
calibration	C4 128 samples, 2048 tokens	fine-tuning 없음
quality	WikiText-2/C4 PPL, 5 zero-shot tasks	metric별 trade-off 가능
hardware	single NVIDIA H100	mobile·Tenstorrent 증거 아님

throughput는 H100 관측치이며 batch·sequence·kernel timing 절차는 명시되지 않는다

Table 1 : 한 quality metric의 우위가 전체 우위는 아니다

Llama-3-8B, rho=.02에서 FE8과 VPTQ를 같은 표 안의 두 metric으로 비교한다

method	bpw	Wiki2 PPL ↓	zero-shot avg ↑
VPTQ	2.27	13.01	60.30
EPTQ E8	2.03	11.27	59.29
EPTQ FE8	2.03	11.95	58.82

FE8의 11.95 PPL은 VPTQ의 13.01보다 낮지만 average 58.82는 60.30보다 낮다

38% faster than FP16은 한 Table 1 조건에서 재계산된다

Llama-2-13B, FE8 rho=.1의 decode throughput과 FP16 row를 비교한 상대 차이다

01	비교 조건	Llama-2-13B, same Table 1
↓		
02	FP16 row	104.4 tok/s
↓		
03	FE8 rho=.1	144.4 tok/s
↓		
04	상대 계산	$144.4 / 104.4 - 1$
↓		
05	결과	0.3831, 약 38.31%

‘up to 38%’는 이 H100 decode 조건의 상대 비교이며, 보편 성능 주장이 아니다

Table 1에는 FE8이 느린 비교도 있다

Llama-2-70B에서는 QTIP와 FE8 rho=.02의 decode throughput을 그대로 읽어야 한다

model	method	rho	bpw	decode tok/s
Llama-2-70B	QTIP	표기 없음	2.00	38.9
Llama-2-70B	EPTQ FE8	.02	2.02	36.4
Llama-2-13B	QTIP	표기 없음	2.01	미측정
Llama-2-70B	FP16	해당 없음	16	미측정: one GPU 불가

표의 미측정은 0이 아니며, 70B QTIP 38.9는 FE8 .02의 36.4보다 크다

Table 2: WSN quality gain, FE8 r=1.75

표의 delta와 괄호 안 absolute value를 혼동하지 않고 읽는다

model	rho	C4 PPL delta	zero-shot delta
Llama-2-7B	.02	-1.44 (10.58→9.14)	+3.0 (58.6→61.6)
Llama-2-13B	.1	-0.37 (8.18→7.81)	+0.9 (66.2→67.1)
Llama-3-8B	.02	-3.44 (18.34→14.90)	+7.3 (51.5→58.8)
Llama-3-8B	.1	-2.33 (16.48→14.15)	+4.2 (56.6→60.9)

마지막 행의 endpoints는 반올림되어 4.3처럼 보일 수 있으나, 논문이 출력한 delta는 +4.2다

Figure 6: rho가 아닌 realized global fraction

adaptive CWP가 만든 실제 전체 critical-weight ratio를 fixed-ratio baseline에도 같은 예산으로 준다

- adaptive CWP는 rho마다 matrix별 tau를 다르게 찾는다
- 그 결과의 전체 preserved fraction을 먼저 계산한다
- fixed-ratio CWP는 그 global fraction을 동일한 ratio로 사용한다
- Figure 6은 같은 realized ratio에서 adaptive가 더 낮은 PPL이라고 보고한다

x축을 rho로 읽으면 같은 preservation budget이라는 비교 조건을 놓친다

Figure 7: radius r sensitivity의 조건

ρ 별로 best r 가 달라질 수 있고, 원고는 $r=1.75$ 를 default로 둔다

- 대상은 Llama-2-13B의 C4 perplexity다
- 각 선은 다른 search ratio ρ 를 나타낸다
- r 1.70부터 1.80 사이에서 PPL이 비교적 안정적이라고 보고한다
- 그림의 best marker는 ρ 마다 하나이며 일반 최적 radius 선언이 아니다

$r=1.75$ 는 이 study의 robust default이며 모든 ρ 의 유일한 최적값은 아니다

원고가 아직 보여 주지 않는 것

결과를 유용하게 읽기 위해 확인된 evidence와 비어 있는 evidence를 분리한다

질문	원고의 근거	말할 수 없는 결론
hardware	single H100 결과	Tenstorrent·mobile 성능
throughput	Table 1 tok/s	batch·sequence·kernel 일반성
reproducibility	future release 언급	즉시 재현 가능
quality	일부 row의 trade-off	모든 baseline·metric 지배

EPTQ의 근거는 공개 원고의 PTQ 설계와 조건부 실험이지 배포 증명은 아니다

EPTQ를 한 문장으로 다시 조립하면

작은 FE8 table만으로 끝내지 않고 WSN과 CWP를 함께 넣어 2-bit PTQ의 quality-efficiency trade-off를 다룬다

01	FE8	same-type factorization으로 4 KiB table
↓		
02	WSN	W-prime의 row·column scale 정렬
↓		
03	CWP	local score로 critical block FP16 보존
↓		
04	compensation	column-wise error propagation
↓		
05	조건부 결과	H100 Table 1·2에서 quality와 speed 보고

핵심은 2-bit code 하나가 아니라 코드북·분포·예외 block의 연결 설계다